SIWES Project Report: Phishing Simulation

Project Title: Phishing Simulation **Program:** SIWES Cybersecurity

1. Project Objective and Scope

The primary objective of this exercise was to demonstrate practical competence in architecting a social engineering campaign. Using the open-source GoPhish framework, the task required designing a targeted phishing simulation from scratch. Strict ethical rules of engagement were observed: the campaign was authorized for creation and local configuration only. It had to be built within a safe lab environment without deploying live network traffic, ensuring zero risk to external organizational infrastructure.

2. Lab Environment and Target Definition

The exercise was conducted on a Kali Linux machine. The GoPhish server was initialized locally, and the administrative dashboard was accessed via a loopback address (`127.0.0.1`). This ensured that all configurations and templates remained entirely contained within the host machine.

To maintain strict ethical boundaries during the demonstration, a custom target group was established using a single synthetic profile associated with demo@example.com. The `example.com` domain is officially reserved by internet authorities specifically for software testing and safe lab environments, ensuring zero risk of accidental spoofing or triggering real-world spam alerts.

3. Sender Identity and Safety Controls

A "Sending Profile" was configured to establish the deceptive origin of the email. In a real-world scenario, attackers use typosquatted domains to trick users, but for this authorized lab, the sender address was also set to the reserved demo@example.com domain to maintain isolation.

Crucially, the SMTP (mail delivery) host was intentionally set to the local loopback port (`127.0.0.1:25`). This acted as the primary technical safeguard for the project; by pointing the outgoing mail to a non-existent local mail server, the framework was physically restricted from routing the test email to the public internet.

4. Educational Landing Page Architecture

A custom web portal was designed using HTML and CSS to act as the destination for the phishing link. Rather than building a malicious login screen to steal passwords, the page was designed as an interactive "Corporate Identity & Access Management" portal. The page was built to immediately notify the user that they were participating in an authorized security simulation and provided a checklist of red flags (such as verifying sender domains). The successful rendering of this code was validated using the GoPhish "Preview" rendering tool, which accurately displayed the trap without requiring live web hosting.

5. Email Pretexting and Tracking Mechanics

An email template was drafted utilizing a "Password Expiration" pretext. This theme leverages artificial urgency, a common psychological trigger used by threat actors to bypass a user's critical thinking. The visual branding of the email was coded to identically match the landing page for a seamless experience. Dynamic GoPhish variables (`{{.URL}}` and `{{.Tracker}}`) were embedded into the HTML to automatically generate a unique tracking link for the target and insert an invisible pixel to log if the email was opened.

6. Campaign Assembly and Live Metrics Dashboard

The final configuration phase involved linking the target group, sending profile, email template, and landing page into a cohesive operation. The tracking URL was bound to the local Kali Linux IP address.

The GoPhish dashboard acts as a command center, designed to provide real-time telemetry on target behavior. It monitors the complete "funnel" of a social engineering attack: Emails Sent, Emails Opened, Links Clicked, Data Submitted. In a live enterprise environment, this data allows a Security Operations Center (SOC) to measure exactly how resilient staff members are against deceptive communication.

7. Final Observations and Safety Verification

Upon officially launching the campaign, the GoPhish system attempted to dispatch the emails but intentionally registered an "Error sending email" alert in the timeline (`dial tcp 127.0.0.1:25: connect: connection refused`).

This error was the planned and expected outcome. It serves as definitive proof of the safety controls put in place. Because the Kali Linux lab environment was safely isolated and did not have an active SMTP mail relay listening on Port 25, the operating system actively rejected the connection. The exercise successfully proved the ability to architect, design, and configure a highly realistic spear-phishing simulation strictly for analytical presentation, remaining securely trapped within the offline environment.

8. Conclusion and Key Takeaways

This SIWES project provided invaluable hands-on experience in both offensive architecture and defensive security awareness. By building a spear-phishing campaign from the ground up, I gained a practical understanding of how social engineering attacks are constructed using spoofed sender profiles, psychological pretexts, and deceptive landing pages. More importantly, this exercise reinforced the principles of ethical hacking and lab safety. Configuring local loopback addresses and encountering the expected Port 25 connection refusal proved the importance of strict environmental isolation. Ultimately, this project demonstrated how security teams utilize these frameworks not to cause harm, but to gather critical telemetry on human vulnerabilities, enabling organizations to build stronger, more resilient security awareness programs.